我的收藏

创建于 2026-01-30 • 0 人关注

添加评论 编辑 删除

2 个内容

一篇讲透 Agent 自进化飞轮怎么搭：评测→记忆→落地→控制

腾讯技术工程，编程话题下的优秀答主

航海家 宇庭Yttrium 等 273 人赞同 ›



作者：yannisyang、ethanytzhou

> 你的 Agent 上线半年了。回头看看——它比第一天聪明了吗？它可能还在犯三个月前就犯过的错。还在重复三个月前就走过的弯路。还在对三个月前就见过的场景表现得像个新手。半年的线上流量喂过去，它一点经验都没攒下来。问题不在模型不够强。问题在于——没有人给它搭一条从"做过"到"记住"到"变好"的路。这篇文章基于对 Anthropic 递归自改进报告、斯坦福 CS329A 课程、EvoAgentX 等开源框架的深度研读，结合业界多个团队已验证的优秀实践，提炼出一套评测→记忆→落地→控制的四齿飞轮方法论——不是某个环节的最佳实践，而是一个完整的、可落地的、经过验证的工程闭环。

开篇：一个被拆散的系统

最近几个月，我观察到一个普遍且致命的现象：**评测、记忆、Self-Improve 这三件事，在大多数团队里是被拆散着做的。**

评测团队搭了一套评分系统，跑完打个分就结束了——分数去了周报，没有去到 Prompt 或 Skill 的改进链路里。记忆团队做了一套向量检索，存了一堆对话历史——但召回的东西噪声太高，Agent 反而被干扰了，最后记忆模块被默默关掉。Self-Improve 团队在探索 Skill 自动生成——但生成出来的 Skill 质量参差不齐，没有评测来筛选好坏，没有版本管理来回滚出错的。

每个团队都在认真地做自己那一块。**但飞轮没有转起来。**

为什么？因为这三件事不是三个独立的功能点——它们是**同一个闭环的不同环节。**评测是闭环的"眼睛"（感知哪里有问题），记忆是闭环的"大脑"（积累可复用的经验），Self-Improve 是闭环的"手脚"（把经验变成实际的改进），人机协作是闭环的"方向盘和刹车"（确保进化方向不跑偏）。拆开来做，每一块都能说"我做了"，但合在一起却不产生复利——**因为环节之间的数据通路是断的。**

我的收藏夹 查看全部 ›

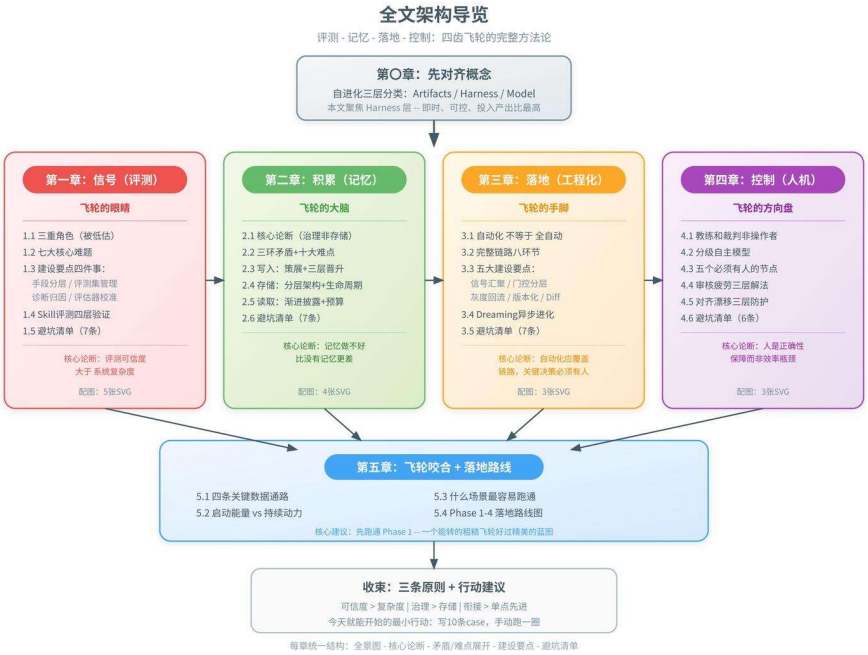
我的收藏
2026-11-10 更新 · 2 条内容 · 0 人关注

LLM
2023-08-03 更新 · 3 条内容 · 0 人关注

炼丹
2022-09-11 更新 · 1 条内容 · 0 人关注

~~来形成一个持续转动的飞轮。~~

读完这一篇，你应该能清楚地知道：自进化系统的每个环节该怎么搭、环节之间该怎么接、落地时该先做什么后做什么、以及哪些地方千万不能踩。



我的核心立场有三个：

> ① **自进化的瓶颈不是任何一个技术点，而是环节之间的衔接。** 评测信号必须能流进记忆和 Skill 更新；更新后必须能被评测验证；验证结果必须能反哺下一轮进化。这条链路断了任何一环，飞轮都转不起来。
> ② **评测的可信度 > 系统的复杂度。** 一个简单但评测可信的系统，远好于一个复杂但评测失真的系统。错误的正反馈会让 Agent 加速开往悬崖。
> ③ **记忆系统的核心不是存储能力，而是治理能力。** 存得多不如治得好——版本控制、主动遗忘、冲突解决、来源溯源，这些"不性感"的治理能力才是决定记忆系统到底帮忙还是添乱的关键。

接下来我会按照飞轮的四个齿——**信号（评测）→ 积累（记忆）→ 落地（工程化）→ 控制（人机协作）**——逐一展开。每个环节我会讲清楚三件事：核心矛盾是什么、应该怎么建设、有哪些坑必须避开。最后一章讲它们怎么咬合成飞轮、以及从零到一的落地路线。

阅读指引（按你的时间选择路径）：

- ⚡ **10分钟速读**：只看每章的 1.1/2.1/3.1/4.1（核心论断）+ 第五章 = 抓住全文精髓
- **按需精读**：你当前最痛的环节是哪个？直接跳到对应章节深入
- **完整阅读**：从头到尾，约30分钟，获得最完整的方法论体系

但在这之前，先对齐一个被频繁混淆的基础概念。

〇、先对齐概念：自进化改的到底是什么？

> *为什么是现在？2026年上半年，自进化从"学术概念"突然变成了"工程热点"——斯坦福开了CS329A课程、Anthropic发布了递归自改进报告和Dreaming机制、多个开源框架（EvoAgentX等）出现、业界多个团队跑通了完整闭环。这个领域正从"能不能做"跨越到"怎么做好"的阶段。*



"自进化"这个词被用得很滥。有人拿 Self-Refine（让 Agent 多改几版输出）叫自进化，有人拿 Agent RL（训模型权重）叫自进化。它们是完全不同的东西。

我把它分为三层——**改什么，就决定了"进化"的持久性和价值**：

第一层：Artifacts 迭代——改输出产物。 代码写了三版越来越好、方案改了几稿越来越靠谱。但任务结束，Agent 本身没有任何改变。下次遇到同类问题，它不会比这次更快。**这不是进化，这是打草稿。** 这一层已经成熟（Self-Refine、Test-time Compute Scaling 等），不是本文讨论的重点。

第二层：Harness 自改进——改 Agent 的配套系统。 记忆、Skill 文件、Prompt、工具配置、工作流——这些改动持久化到文件或数据库，跨会话生效。改一次，后续所有任务都受益。**这是当前最具性价比的进化方向，也是本文的主战场。**

第三层：Model 进化——改模型参数本身。 通过 Agent RL、自对弈微调将经验写入权重。持久性最强，但成本最高、风险最大。目前仍在研究阶段——MiniMax M2.7 和业界多个团队的调研都在探索这条路，但距离生产落地还有距离。

为什么 Harness 层是主战场？ 四个字：**即时、可控。** 改完 Skill/Prompt 立刻生效，不需要重训模型；改错了一键回滚，不会造成不可逆的损害。实测数据也在佐证——已有落地案例通过 Harness 层进化，迭代次数减少 70%、Token 消耗降低 80%+；在 Agent Memory 场景中 Token 节省超 60%——**这些收益全部来自"改配套系统"，一个模型参数都没动。**

这三层之间并非割裂。Harness 积累的经验可以变成训练数据改进 Model；更强的 Model 又能产出更好的 Artifacts。但在当下这个时间点，**Harness 层是投入产出比最高的着力点**——接下来四章的讨论都聚焦于此。

一个具体的例子帮助理解三层的区别：

假设你的Agent在处理"日期格式转换"时经常出错。三层分别怎么解决这个问题？

- **Artifacts层**：让Agent在本次任务中多试几次、自我纠错——这次做对了，但下次遇到同样的问题它不会更快。
- **Harness层**：写一条Skill——"遇到日期格式转换时，先识别源格式再用对应的转换函数"——持久化保存，下次遇到直接follow这条Skill，不再犯错。
- **Model层**：用大量日期格式转换的正确样本微调模型——模型本身"学会"了这件事。成本最高，但也最根本。

对于绝大多数团队来说，**写一条Skill（Harness层）**就能解决问题——不需要微调模型。这就是为什么Harness层是主战场。

一、信号——评测不是打分，是整个飞轮的信号源

> *飞轮的第一齿。一个看不见自己缺陷的系统，不可能进化。*



1.1 我的核心论断：评测在自进化中的角色被严重低估了

大多数团队把评测当作"上线前打个分"——跑一轮，看个数字，报告一下。

在自进化系统中，评测承担的角色远不止于此。 它至少有三重职责：

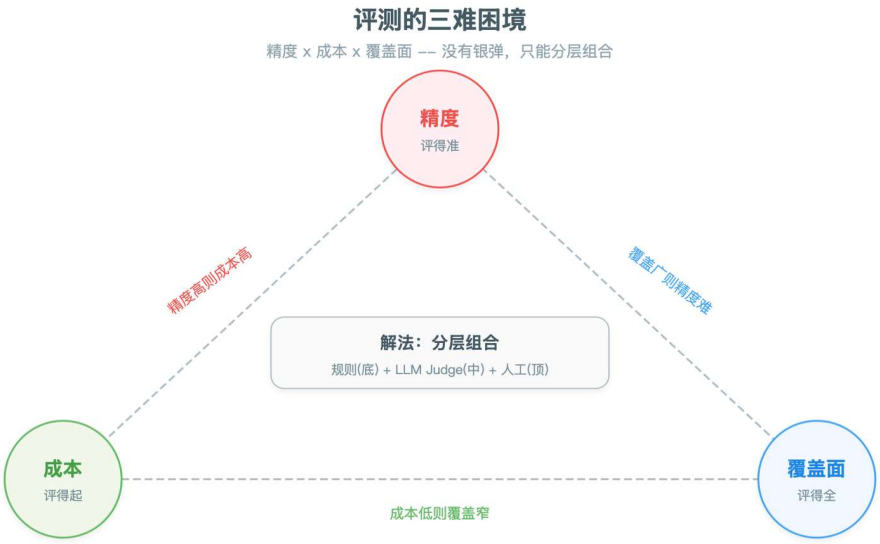
- **方向指引**——告诉系统"哪里有问题"，决定进化朝什么方向走
- **质量门控**——更新 Skill/Prompt 前验证"改了是否更好"，防止退化
- **经验筛选**——判断一次执行结果是"好经验"值得沉淀，还是"坏经验"应该丢弃

为什么要强调这三重角色？因为**传统评测只需要承担"质量门控"一个职责**——上线前跑一轮，达标就放行，不达标就打回。但自进化场景下，评测多了两个全新角色：它要告诉系统"下一步该往哪改"（方向指引），还要判断"这次执行产生的经验值不值得存下来"（经验筛选）。这两个新角色意味着——**评测信号会直接流入记忆和 Skill 更新链路，影响 Agent 的长期行为。** 传统评测打错分顶多是"这次上线了一个不够好的版本"；自进化评测打错分是"Agent 学到了错误经验并持续污染后续所有任务"。

这三重职责意味着：**评测的质量直接决定了记忆的质量、Skill 更新的方向、以及整个飞轮是正转还是反转。**

如果评测信号失真——错误的正反馈会让 Agent 学到错误经验，写入记忆和 Skill，污染后续所有任务。**这不是"不够好"的问题，是"加速犯错"的问题。** 所以我才说：**评测的可信度 > 系统的复杂度。**在搭任何自动化之前，先确保你的评测信号是准的。

1.2 评测体系的七大核心难题



在这个三难困境之下，实践中会碰到七个具体的难题。我把它们按"最致命→最常见"的顺序排列：



难题一：弱评估器——最致命的问题，排在第一位。

用 LLM 给 LLM 打分，听起来很优雅，但评估器自身可能有系统性偏差——偏好长文本（详细但冗余的答案得高分）、偏好特定格式（有编号列表的答案比段落式的得高分）、偏好表面正确性（引用了数据但数据是编造的也得高分）。

这种偏差的可怕之处在于：它**不是随机噪声，而是系统性的。**一旦评估器有偏好长文本的偏差，Agent 会在多轮进化中"学会"把答案写得越来越长——不是因为长更好，而是因为长能骗过评估器。**错误的正反馈比没有反馈更可怕——它让Agent"自信地学到错误经验"，相当于加速开往悬崖。**

难题二：评什么维度——Agent 的输出不是一个单一答案。

传统任务评测（如NLP benchmark）评的是一个答案对不对。但 Agent 的输出是"推理过程 + 工具调用序列 + 中间结果 + 最终答案"的组合体。只看最终答案会漏掉大量问题：

- 过程中产生了幻觉但碰巧最终答案对了——这种"虚假通过"下次大概率会翻车
- 工具调用冗余——做对了但消耗了5倍Token，效率极低
- 推理链有错误但因为模型记忆力强"记住"了正确答案——换个问法就垮

只评结果不评过程，盲区里的问题永远不会被发现和修复。

难题三：评什么粒度——精度和成本的正面博弈。

评"整个任务成功/失败"（粗粒度）——便宜，但只知道"错了"不知道"错在哪"。评测说"60分"，到底是 Prompt 写得不清楚？Skill 逻辑有Bug？还是API调用参数根本就写错了？粗粒度评测无法回答这个问题，导致后续修复只能"瞎改"。

评到"每个step/tool call/决策节点"（细粒度）——精确，能定位到具体哪一步出了问题，但成本指数级增长。实测一轮完整的细粒度评测需要几百到上千次LLM调用。

粒度的选择直接决定了归因的精度，而归因的精度决定了修复能否精准命中。 这里没有标准答案——需要根据场景在两者之间找平衡。我的建议是：日常用粗粒度做快速筛选，对失败的case再用细粒度做深度归因。

难题四：开放式任务难量化。

编程任务可以跑测试、SQL可以对比结果——有确定性的对错标准。但写作、策略规划、方案设计这类任务怎么评？什么叫"好的方案"？十个资深工程师可能给出十个不同的判断。

人工审查是最终权威，但不可扩展——一天审10条还行，一天审1000条不现实。纯LLM Judge又有上面说的弱评估器偏差。**这限制了自进化在开放式场景的适用范围**——也是为什么当前最成功的

分"，排序比打分更稳定）；或者**AI初筛+人复核**的流水线（AI先过滤明显不合格的，人只审有争议的10-20%）。

> **中间小结：** 前四个难题的共性是——它们都在问"评测信号本身是否可信"。弱评估器、维度缺失、粒度不对、标准缺失——本质上都是**信号质量问题**。接下来的三个难题则是另一类问题：**信号的新鲜度和成本**。

难题五：评测集漂移——数字漂亮但用户骂街。

业务在变、用户需求在变、环境在变，但评测集往往半年不更新。Agent在旧集上"刷出高分"，你看着报告觉得一切很好——但线上用户的真实体验可能在持续下降。

评测集漂移的隐蔽性在于：**没人会在评测全绿的时候去怀疑评测本身有问题。**等你发现时，可能已经在错误方向上跑了很远。

难题六：预算公平性——你的"进化"可能只是多花了钱。

这是一个容易被忽视但很深刻的问题。很多"自动进化"的性能提升，可能仅仅来自多花了计算预算——比如多次采样然后择优（Best-of-N），或者让Agent重试几次取最好的。这种提升一旦关掉额外预算就打回原形。

真正的进化应该是：在相同的Token预算下，Agent也比之前做得更好。 如果新方案消耗3倍Token才比旧方案好10%——这不是进化，是用钱砸分。评估必须在预算对等条件下进行，否则就是自欺欺人。

难题七：评测成本——每次更新都跑全量评测吃不消。

一轮完整评测可能需要几百次LLM调用。如果每次Skill/Prompt修改都跑全量——Token成本很快失控。但不跑又怕引入回归（原来能做对的做错了）。

这个成本约束直接限制了迭代频率。如果评测太贵导致你一周只能跑一轮——飞轮就转得很慢。解法是分层评测：每次改动跑轻量级回归（核心case），定期跑全量评测（全集）。

1.3 建设要点：怎么搭一套可信的评测体系



一套完整的评测体系需要回答四个问题：**用什么手段评 → 评测集怎么管理 → 评完怎么处理结果 → 怎么确保评测器本身可信。** 下面逐一展开。

建设要点一：评测手段分层组合，把成本花在刀刃上。



核心原则：**能用便宜方法覆盖的场景，不要用贵的方法。** 从底层到顶层：

底层：规则/自动化校验。 代码能跑、测试通过、SQL结果匹配、格式正确、响应时间达标——零成本、100%客观、无偏差。这是评测体系的基石，也是大多数团队没有充分利用的层级。很多本可以用规则校验的维度（如格式是否合规、字段是否完整），却在用LLM Judge来评——纯粹浪费。

中间层：LLM-as-Judge + 对照实验。 适用于开放式任务和需要语义理解的评测维度。几条关键原则：

- **生成模型和评估模型必须分离。** 让Agent评价自己的输出，会触发LLM的自洽偏差——它倾向于认为自己是对的。用一个独立的模型实例来评估，或者用不同的模型。
- **评估 temperature 设为 0。** 确保对同一份输出的评价是一致的、可复现的。否则跑两次评测得到不同分数——你信哪个？
- **输出必须结构化。** 用 Pydantic 或 JSON Schema 定义严格的评估维度和打分标准。不要让评估器"自由发挥"——它会在不同维度之间随意漂移。
- **分维度独立评估。** 不要让一个评估调用同时评判"正确性+完整性+格式+效率"——维度越多，每个维度的评估质量越差。拆开来，每个维度独立评。

顶层：人工抽样 + 元评测集。 定期从评测结果中随机抽样，人工做二次评审——比较人的判断和LLM Judge的打分是否一致。不一致率超过阈值就要校准评估器。

同时维护一小批**"元评测集"**——几十条明确标注为"绝对正确"和"绝对错误"的样本。定期拿出来检查评估器是否还能正确识别。如果评估器连这些极端case都判错了——它的所有打分都不可信，系统应该告警。

建设要点二：评测集的工程化管理——这是最容易被忽视的关键。

大多数团队的评测集管理非常随意：一份文件，想到了就加几条，半年不更新，所有环节共用同一份。这会导致严重的过拟合——多轮迭代后系统在这份集上"刷出高分"，但面对新问题直接垮。

我的建议是**评测集三分法**，严格职责分离：



- **train 集**——只用于诊断阶段：告诉系统"哪类问题失败率最高"、"失败模式是什么"。诊断信息可以暴露给LLM（用于生成修复方案），但不用于"判断谁更好"。
- **val 集**——只用于筛选阶段：判断候选变体"是否比基线更好"。**绝不暴露给生成修复方案的LLM**——否则LLM会"记住"val集的答案。
- **test 集**——只用于最终验证：独立于前两者，在决定是否上线前做最后一道把关。test集的存在是为了捕捉"在val上过拟合但对新问题无效"的情况。

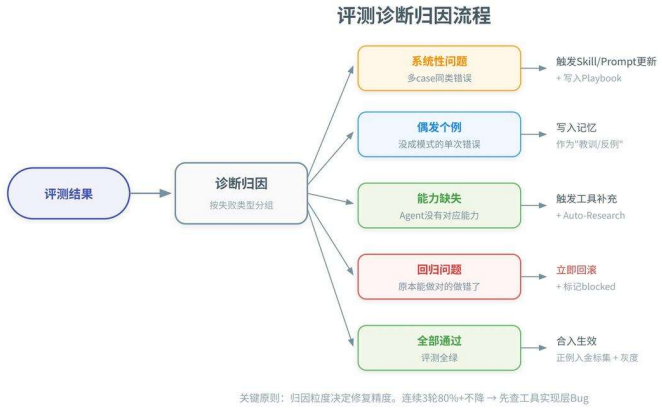
实操指导——怎么划分?

- **比例建议**：train 50-60% / val 20-30% / test 15-20%。train可以大一些（诊断需要足够的失败样本来发现模式），test要保持精简但覆盖面广。
- **划分维度**：按"问题类型"分层抽样，确保每个集里各类型都有覆盖——不能出现"某类问题只在train里有，val和test里完全没有"的情况。
- **更新节奏**：val集建议每2-4周更新一批（替换20-30%的旧样本）；test集更稳定，每1-2月更新一次；train集可以持续追加线上新失败样本。

建设要点三：评测的终点不是打分，是诊断归因。

分数告诉你"60分"，但不告诉你为什么。**没有归因的评测，就像体检只告诉你"不健康"但不告诉你哪里不舒服——你知道有问题，但完全不知道该改什么。**

评测完的下一步必须是诊断归因——按失败类型分组、分析失败模式、定位根因。然后根据根因的类型，走不同的处理链路：



系统性问题（多个case出现同类错误模式）→ 触发 Skill/Prompt 更新，同时写入 Playbook 作为"已知方向"供后续实验参考。

偶发个例（只有一两个case出错，没有成模式）→ 写入记忆系统作为"教训/反例"，供后续遇到类似场景时召回。

能力缺失（Agent根本没有对应的工具或知识来完成某类任务）→ 触发工具/能力补充需求，可以用 Auto-Research 搜索业界方案。

回归问题（原本能做对的事，更新后做错了）→ **立即回滚**到上一版本，标记为 blocked，人工介入排查。

全部通过 → 新版本合入生效，正例加入"金标集"（后续可作为记忆参考或评测集的正样本），进入灰度观测期。

归因的粒度决定修复的精度。 粗归因——"这个 Skill 不好"——导致大面积重写；精细归因——"参数提取阶段的正则表达式漏了 `YYYY-MM-DD` 这种格式"——才能做到精准修复而不影响其他逻辑。

一条血泪教训：如果某个场景连续3轮失败率80%+不下降——**先停下来查工具实现层**（API参数错误、服务端返回异常、数据根本没回来），而非继续在配置层打转。配置层的进化解决不了工具层的Bug——实践中出现过20+轮无效迭代、最终发现根因在工具层而非配置层的案例。

都不可信——但你如何发现这一点?

我建议四种互补机制:



1.4 Skill 评测的特殊性——证明 Skill "真的有用"比想象中难

Skill 是 Harness 层进化的核心产出物——它把经验沉淀为可复用的操作指南。但Skill写出来之后,怎么证明它"真的有用"而非"碰巧存在"?

这个问题比看起来复杂得多。**很多 Skill 看似有效,实际上只是和模型能力碰巧重叠了——Skill 的存在只是"恰好和Agent本来就会做的事一样"。**



我认为 Skill 评测至少需要通过**四层验证**,才能说"这个Skill真的有价值":

第一层:对照实验——模型本来就会做吗?

最基本的验证:设计有Skill和无Skill两组对照实验,跑同一批评测case。如果无Skill时Agent也能做对——那Skill的"贡献"是零。很多团队跳过了这一步,导致积累了大量"看起来有用但实际无用"的Skill——它们只是在浪费上下文窗口。

第二层:难度校准——任务是否太简单?

如果评测case本身非常简单(模型不需要任何额外知识就能做对),那无论有没有Skill结果都是对的——对照实验看不出差异。需要设计**有一定难度梯度的评测case**——让模型在"无Skill时能做对一部分但做错一部分"的水平上,才能看出Skill的增量价值。

第三层:轨迹追踪——运行时真的调用了Skill吗?

一个常见的假象:Skill存在于上下文中,Agent最终给出了正确答案,但实际上Agent**根本没有follow Skill里的步骤**——它是靠自身能力完成的,Skill只是占了位置没被使用。

验证方法:在运行时追踪Agent的执行轨迹(tool calls、推理链),检查是否出现了Skill中定义的关键步骤。如果Skill写了"先调用API A查询X,再用结果调用API B",但实际轨迹里Agent直接调了API C——说明Skill没有被真正使用。

第四层:路径验证——结果对了,但路径对吗?

举个例子：Skill要求"查询完结果后必须做一次格式校验"。Agent查了结果但跳过了格式校验——这次碰巧数据格式是对的，所以最终答案正确。但下次如果数据格式有问题，Agent就会给出错误输出。**路径不对意味着下次会翻车——即使这次结果对了。**

Skill 评测的工程化建议：

- 每个 Skill 上线前必须跑对照实验（有/无Skill两组，验证增量价值）
- 评测case要有难度梯度——太简单的case看不出差异
- 引入轨迹评测（trace-level evaluation）——不只看最终输出，还要看执行路径
- 定期复评——Skill可能因为环境变化而过时（API改了、需求变了），需要定期重新验证

这套方法论在业界多个团队的 Skills 轨迹评测实践和综合评测平台（覆盖结果质量+过程质量+安全质量三维度）中都有成熟落地。

1.5 评测体系避坑清单

评测体系避坑清单

坑	严重度	为什么会踩	正确做法
单一评估方法	高	觉得"有评测就行"，一种方法覆盖所有	分层组合：规则+Judge+人工
评测集全场景共用	高	图省事，一份集子到处用	三分法：train/val/test职责分离
不校准评估器	高	默认LLM Judge永远可信	定期人工抽样校准+元评测集监控
只评结果不评过程	中	Agent最终答案对就行了	引入轨迹评测，检查工具调用和推理链
评测集长期不更新	中	"上次跑还行"，没人维护	线上失败样本定期回流，每月更新
忽视预算公平性	中	多花3倍Token才好10%，以为是进化	控制变量：同Token预算下对比
配置层无限打转	低频致命	失败率不降但一直在改Prompt	3轮80%+不降→先查工具实现层

评测解决了"发现问题"和"判断好坏"——但发现了问题之后呢？好的经验怎么被记住、坏的教训怎么不被遗忘？这就是下一章要解决的问题。

二、积累——记忆系统的核心不是存储，而是治理

> *飞轮的第二齿。评测给了信号，但信号会随风消散——除非有一个系统把它"记住"。*



2.1 我的核心论断：记忆做不好，比没有记忆更差

我观察到的多个团队在尝试给 Agent 加上长期记忆后，因为检索噪声过高，最终降级回无状态模式——记忆模块被默默关掉。与此同时，具备有效长期记忆的 Agent，用户体验和留存率显著优于无记忆 Agent。

这组矛盾数据说明一件事：**记忆系统的难度被严重低估了。** 不是"加一个向量库存起来就行"——存了之后召回不准，注入上下文后 Agent 被干扰，还不如不记。

记忆系统真正要解决的不是"存储"问题——存储是最简单的部分。真正的挑战在**治理**：什么该存、什么该忘、怎么确保存的是对的、怎么在需要时精准召回、怎么不占用太多上下文窗口。

把记忆系统当作一个"治理系统"而非"存储系统"来设计，是我认为最重要的认知转变。 就像代码仓库——Git 的价值不在于"能存代码"，而在于版本控制、分支管理、冲突解决、审核流程这些"治理能力"。记忆系统也一样：**治理能力决定了它到底是帮忙还是添乱。**

2.2 记忆的三环矛盾与十大难点



记忆系统需要同时解决**写入、存储、读取**三个环节的矛盾。我把每个环节的具体难点展开如下：

写入环——沉淀什么是有效的?

① 价值判断难。 任务执行中产生大量中间信息——Tool call的返回、推理过程、搜索结果——绝大部分是噪声，但真正有价值的经验可能藏在某个不起眼的细节里（比如"这个API在凌晨3点会限流"）。Agent如何自主判断什么值得永久保存?

② 正负反馈不对称。 任务成功时，你知道"做对了"但不知道"因为什么做对"——是Prompt好？还是碰巧数据质量高？归因难导致写入的经验可能归因错误。

③ 记忆粒度选择。 存得太粗（"用方案A成功了"）无法复用到类似场景；存得太细（每个tool call完整返回）信息过载。粒度取决于场景，没有统一标准。

④ 隐性知识难捕获。 最有价值的经验往往是"什么不能做"——"这个API的timeout不要用默认值"、"这种格式的数据不要直接转JSON"。Agent很难自主识别这类"负空间"知识。

务。**错误蔓延速度远快于修复速度**——这是记忆系统最危险的失败模式。

⑥ **时效性衰减。** 半年前写入的信息可能已经过时——但没人清理。Agent基于过期知识做决策，产生"高置信的错误输出"。

⑦ **记忆冲突。** 不同来源、不同时间写入的记忆可能互相矛盾。没有冲突解决机制时，Agent可能随机引用其中一条。多Agent并发写入时问题更严重。

⑧ **存储成本失控。** 没有遗忘策略的记忆系统，存储量只会线性增长——记忆越多、检索越慢、噪声越高。

> **中间小结：** 写入环的四个难点本质是"什么值得存"；存储环的四个难点本质是"存了之后怎么不腐烂"。它们的共性是——**不治理的记忆不如没有记忆。**接下来的读取环则是另一个维度的问题：即使存储治理得很好，怎么在需要时精准召回、且不撑爆上下文？

读取环——怎么用不出问题?

⑨ **检索噪声。** 向量相似≠语义相关。"帮我查一下订单状态"可能检索出"帮我取消订单"——向量距离近但语义完全不同。注入错误的"相关记忆"比不注入更糟。

⑩ **上下文预算分配。** 记忆注入占用有限的上下文窗口。注入太多→挤压工作空间+分散注意力；注入太少→漏掉关键背景。没有自适应机制是通病。

2.3 建设要点一：写入——"策展而非全存"

核心原则：**Agent不是录音机，而是策展人。**

正信号触发写入： 只有收到明确的正信号时才写入长期记忆——用户确认/评测通过/任务成功。没有信号的中间过程默认不存。

失败也有价值，但存法不同： 失败任务同样提取经验，但标记为"反例/陷阱"——"这条路走不通，因为XX原因"。一个明确的反例比一个模糊的正例更有价值。

三层晋升机制（来自 self-improving-agent Skill，SkillHub高下载量热门Skill）：



精妙之处：先用低门槛解决"忘记记录"（层级1），再通过门槛层层筛选把有价值的内容往上升级。越往上质量越高、噪声越少。

非对称淘汰设计： 好经验强化速度(+0.05)远慢于坏经验淘汰速度(-0.12)——淘汰是强化的2.4倍。逻辑：**一条错误记忆的伤害 > 一条正确记忆的收益。** 宁可慢一点积累好经验，也不能让坏经验扩散。

2.4 建设要点二：存储——"分层架构 + 生命周期管理"



分层架构已成为业界共识（Hermes Agent 56K+ Star、多个开源 Agent Memory 框架等验证）：

分层记忆架构：三层对照

层级	认知对应	内容	生命周期	使用频率
程序性记忆(Skill)	"怎么做"	操作步骤+触发条件+注意事项	长期，版本化	匹配即用
语义记忆(Facts)	"知道什么"	环境知识、项目约定、偏好	中长期，可更新	日常默认注入
情景记忆(Episodes)	"经历了什么"	历史对话、操作轨迹	短中期，频率衰减	需要时检索

分层的核心价值：大多数系统卡在"原始对话→向量库"的单层模式——什么都存一个池子，全凭向量相似度召回。**分层让存储有结构、召回有层次**——默认用顶层（轻量高密度），需要时才钻取到下层（详细有证据）。更精细的做法是细化为四层（L0 原始对话→L1 原子事实→L2 场景块→L3 用户画像），核心是"默认 L3，按需钻取 L1/L0"。

生命周期管理（记忆不是静态的）：

- **版本控制**：每条记忆带版本号+来源标记（哪个会话/谁触发/因何写入）
- **演化能力**：记忆可以沿时间线更新、合并、分裂、衰减
- **主动遗忘**：TTL 过期/使用频率衰减/被负反馈标记后快速淘汰
- **冲突解决**：后入优先+人工仲裁高冲突项+乐观锁防并发

一句话总结："记忆首先是一套治理系统。" 没有治理，错误信息传播的带宽也会提高。

2.5 建设要点三：读取——"渐进披露 + 严格预算"



我总结的**"从粗到细、按需钻取"模型**：

Step 1：默认只注入顶层。 会话开始只注入 Persona+Skill 前言——约 500-800 token。这是"你需要知道的最小背景"。

Step 2：按需检索中层。 任务到来时，根据语义检索相关 Facts 和 Scenario——三路并行（语义向量+全文关键词+精确匹配），融合排序取 Top-K。

Step 3：需要时才钻取底层。 只有 Agent 执行中遇到困惑或需要证据时，才通过 node_id 回溯到原始对话——平时不加载。

严格预算控制：

- 分层 Token 预算：Skill≤2000, Facts≤800, Profile≤500
- 单条上限：每条≤200 Token（强制精炼）

一个反直觉的发现（Anthropic Dreaming 2026.5）：**记忆建模为文件系统，让Agent用Bash/grep检索管理，比硬造专用API更有效。** Agent本就擅长操作文件——ls/grep/cat是它最自然的交互方式。

符号化压缩是另一个值得关注的方向：用Mermaid编码任务状态，完整日志卸载到外部，Agent在符号图上推理、需要时通过node_id检索——实测**Token节省超60%，任务通过率显著提升。**

2.6 记忆系统避坑清单

记忆系统避坑清单

	坑	严重度	为什么会踩	正确做法
高	全存全塞	高	"先存起来以后再说"	策展式存储+三层晋升门控
高	无遗忘机制	高	"数据是资产"的惯性思维	TTL+频率衰减+负反馈快速淘汰
中	造专用记忆API	中	觉得需要专门的读写工具	建模为文件系统，用Bash/grep管理
中	并发写入无保护	中	多Agent同时写，没想到会冲突	乐观锁(哈希对比)+权限分层
中	不记录来源	中	只存内容不存元数据	每条带session_id/trigger/reason
低	忽视时效性	中低	写入时对的，半年后没人管	TTL+定期复验+使用时校验
低	单层向量库	中低	觉得"有向量检索就够了"	分层：顶层默认注入+底层按需钻取

评测告诉我们"哪里有问题"，记忆帮我们"记住经验和教训"——但光知道问题、光有经验还不够。经验要变成实际的改进才有价值——从"知道该怎么做"到"真的改好了并安全上线"之间，还有一段不短的路。这就是下一章要讲的：落地工程化。（注意：记忆解决的是"存什么、怎么治理、怎么召回"；落地解决的是"改什么、怎么生成修复、怎么安全上线"——它们的核心矛盾不同，所以分开讲。）

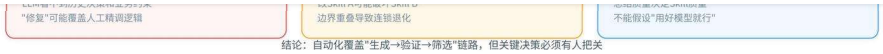
三、落地——从"知道问题"到"产出修复"之间有多远？

> *飞轮的第三齿。评测发现了问题，记忆积累了经验——但从"知道"到"改好"之间，有一个比想象中大得多的gap。*



3.1 我的核心论断：自动化 ≠ 全自动

很多人对"自进化"的想象是——Agent自己发现问题、自己改、自己上线——全程无人。**这在当前阶段是危险的幻想。** 原因有三：



第一，LLM生成的修复质量不可控。 它能看到代码和评测输出，但看不到代码背后的历史决策、业务约束、API规范。结果是——"修复"了问题A，覆盖掉了上次人工精心调整的逻辑。

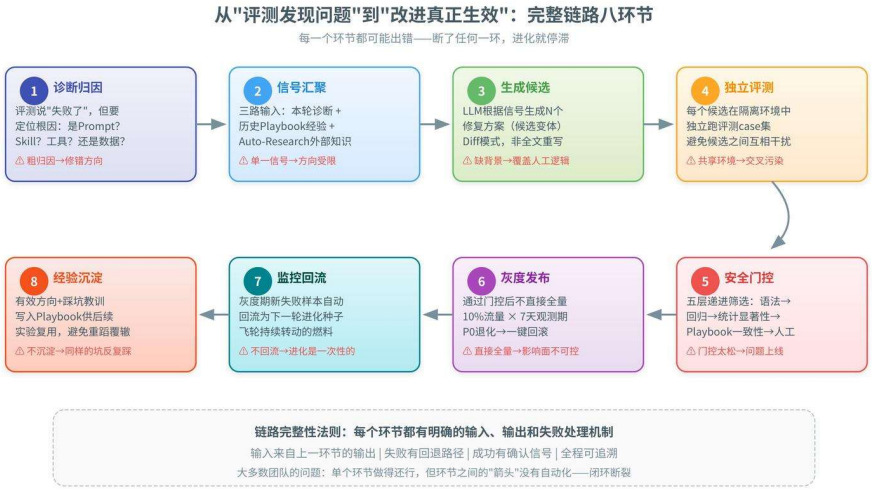
第二，单点修复缺乏全局视角。 改了Skill A解决了问题X，但悄悄破坏了Skill B的触发逻辑——两个Skill在某些查询上有边界重叠，单改一侧只是把问题"推"到另一侧。

第三，"总结经验→生成Skill"本身需要专门优化。 业界调研指出：几乎所有系统默认用大模型来总结，但SkillOS研究证明**训练过的小模型策展人效果优于冻结的大模型策展人**。总结质量决定Skill质量，这个环节不能假设"用好模型就行"。

所以我的观点：**自动化覆盖"生成候选→评测验证→门控筛选"链路，但关键决策必须有人把关。** 不是效率妥协，是正确性保障。

3.2 完整链路的八个环节

从"评测发现问题"到"改进真正生效"，完整链路远比想象中长。我把它拆为八个环节——**每个环节都有明确的输入、输出和失败处理，断了任何一环，进化就停滞。**



逐一展开每个环节的核心要点和典型出错方式：

① 诊断归因——从"失败了"到"为什么失败"。 评测告诉你"60分"，但不告诉你为什么。诊断归因要把失败case按类型分组、识别出模式、定位到具体是哪个环节的问题（Prompt写得不清楚？Skill逻辑有Bug？工具API返回异常？数据本身就有问题？）。**归因的粒度决定后续修复的精度**——粗归因（"这个Skill不好"）只能大面积重写；精细归因（"参数提取的正则漏了一种日期格式"）才能精准修复。

② 信号汇聚——修复方向不能只看"这次出了什么错"。 单次评测结论只是信号的一部分。有效的修复方案需要三路输入：本轮诊断结论、历史Playbook（之前什么改法有效/踩了什么坑）、以及外部知识（业界有没有更好的做法）。只看单一信号容易陷入局部最优——重复尝试同样的思路，导致平台期停滞。

③ 生成候选——LLM产出修复方案的质量是瓶颈。 根据汇聚的信号，LLM生成N个候选修复方案。关键约束：**用Diff模式（只输出需要修改的部分），而非全文重写**——全文重写风险极大，容易覆盖无关的正确逻辑。同时要注入背景知识（API规范、历史决策原因），否则LLM缺乏上下文会做出"看起来合理但实际有害"的修改。

④ 独立评测——每个候选必须在隔离环境中验证。 多个候选不能在同一个环境里跑——它们可能互相干扰（一个候选改了配置影响了另一个的测试结果）。每个候选独立跑评测case集，产出独立的评分和诊断报告。这一步的成本最高（N个候选×完整评测），也是为什么前面的"生成候选"环节需要控制N的大小。

自动过滤95%低质变体，人只审"大概率有效"的少量候选。

⑥ 灰度发布——通过门控≠直接全量。 小流量（10%）先跑7天观测期。监控核心指标（成功率、Token消耗、延迟、用户反馈）。如果出现P0级退化——一键回滚，不需要等人判断。灰度是最后一道防线——它能捕捉到评测集覆盖不到的长尾问题。

⑦ 监控回流——灰度期间的数据是下一轮进化的燃料。 灰度观测期内新积累的失败样本，自动进入下一轮进化的种子池。这意味着：**上线不是终点，而是下一轮进化的起点。** 没有回流机制的系统，进化是一次性活动——做完一轮就停了。有回流机制的系统，飞轮才能持续转动。

⑧ 经验沉淀——本轮的有效方向和踩坑教训必须记录。 这一轮尝试了哪些方向？哪些有效？哪些无效？为什么？——都写入Playbook（进化流水线专属的经验存储，和第二章讲的Agent通用记忆系统互补但独立——Playbook服务于"怎么改Agent"，通用记忆服务于"Agent怎么做任务"）。下一轮实验时，LLM可以参考Playbook避免重复踩坑、优先尝试已验证有效的方向。**没有沉淀的系统，每一轮实验都是从零开始——同样的弯路反复走。**

> **一个更深层的洞察：** 这八个环节本质上是在回答一个问题——**怎么让"改进Agent"这件事本身也变成一个工程化的、可靠的流程？** 传统软件工程用了几十年建立起CI/CD的方法论来保障"改代码"的质量和安全。Agent自进化面临的挑战是一样的——只不过被修改的对象从代码变成了Prompt/Skill/记忆。**自进化的工程化，本质上是"Agent CI/CD"。**

3.3 关键建设要点

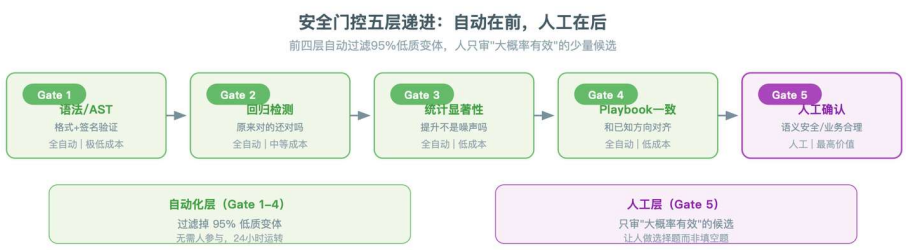


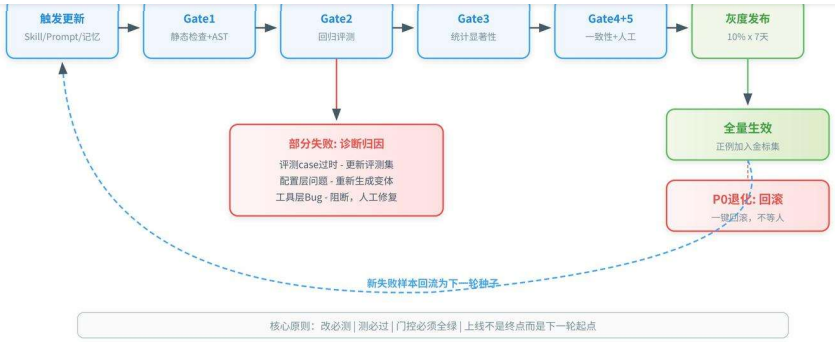
建设要点一：三路信号汇聚。 修复方向不能只靠"这次评测哪些case失败了"——信息量不够。至少汇聚三路：

- **本轮评测诊断**——"这次出了什么问题"
- **历史Playbook**——"以前什么改法有效/踩了什么坑"
- **Auto-Research外部知识**——"业界有没有更好的做法"

第三路解决**平台期停滞**：手写方向用完后系统进入"怎么改都提升不了"的状态。自动搜索论文/开源项目发现新方向，能突破瓶颈。实践中验证过：系统自己发现了两条有效方向——都不在初始配置里。

建设要点二：安全门控分层——自动在前，人工在后。





前四层过滤掉95%低质变体——到人手上的是"大概率有效"的候选。**让人做选择题而非填空题。**

建设要点三：灰度+回流——让进化持续发生而非一次性活动。

通过门控≠直接全量上线。我建议的标准流程：

10%流量灰度 × 7天观测期。 监控四个核心指标：任务成功率（不能下降）、Token消耗（不能暴涨）、延迟（不能恶化）、用户反馈（有无投诉）。任何一项出现P0级退化→一键回滚，不需要等人判断——**自动回滚是灰度机制的灵魂**，如果回滚还需要人来决策，半夜出问题就没人兜底了。

回流机制是飞轮持续转动的关键：灰度期间新积累的失败样本不是简单地"记录一下"——而是**自动进入下一轮进化的种子池**，成为下一轮诊断归因和修复方案生成的输入。这意味着：上线不是终点，而是下一轮进化的起点。没有回流机制的系统，进化是一次性活动——做完一轮就停了。有回流的系统，飞轮每转一圈都比上一圈有更多信息。

建设要点四：版本化一切——出了问题能追溯能回滚。

这不是"锦上添花"的工程规范——这是自进化系统的**生存基础设施**。因为自进化意味着系统在持续修改自己——如果修改不可追溯、不可回滚，出了问题就是灾难。

Skill/Prompt/记忆的每次变更都必须有：

- **版本号**——可以随时回到任何一个历史版本（类似Git的commit hash）
- **变更日志**——谁改的、为什么改、具体改了什么（diff可见）
- **来源标记**——这次变更是由哪次评测触发的？是自动生成还是人工修改？
- **关联评测结果**——这次变更在什么数据上验证过？通过率是多少？

有了这四项，出问题时的排查链路是清晰的：线上异常→定位到哪个版本引入的→查看这个版本的变更内容→查看它关联的评测结果→决定回滚到哪个版本。没有版本化的系统，出问题后只能"再改改试试"——盲人摸象。

建设要点五：Diff模式——修复时只改需要改的部分。

让LLM全文重写一个Skill文件——风险极大：

- 容易**覆盖无关逻辑**（LLM不知道某个条件分支是上次人工精心调整的）
- 改动**无法review**（全文输出，reviewer看不出到底改了什么）
- 出问题**无法精准回滚**（只能回滚到改之前的整个文件）

更好的做法是**Diff模式**：LLM只输出需要修改的片段（"把第X行的正则从A改成B"），框架负责patch合并。好处是：

- 可精准回滚 一个patch对应一个改动，哪个有问题撤哪个

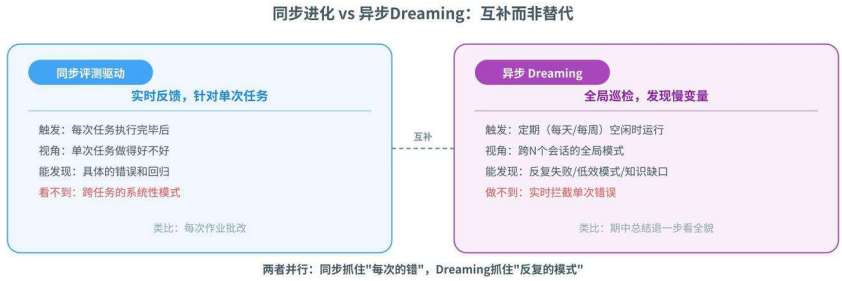
多文件联动修改需要额外注意：改了Skill A的输出格式，可能需要同时改Skill B的输入解析。这种情况下需要**AST签名验证**——确认跨文件的函数调用、数据接口签名没有因为单侧修改而不兼容。来自实践中的血泪教训：多Skill联动改写导致的隐蔽回归是最难排查的问题之一。

3.4 Dreaming：异步的"做梦"式进化——一条被低估的互补路径

前面讲的都是**同步进化**——评测发现问题→立即修复→验证→上线。但还有一类问题是同步进化抓不到的：**跨会话的系统性模式。**

比如：Agent在过去100次会话中，有30次在"时间格式转换"这个环节出了小问题——每次都不完全相同，单次看都是"偶发个例"，但放在一起看是一个系统性短板。同步评测一次只看一个任务——看不到这种全局模式。

Anthropic Dreaming 2026.5 提出的**Dreaming**机制正是解决这个问题的：



核心思路： 一个专职的异步进程（不在任务执行时运行，而是像"睡觉做梦"一样在空闲时运行），定期审阅一批历史会话轨迹，寻找以下模式：

- 反复出现的失败模式（多个会话都在同一类问题上犯错）
- 低效模式（任务最终成功了，但中间走了很多弯路、消耗了不必要的Token）
- 知识缺口（反复需要查外部资料的领域——说明应该沉淀为内置知识）

输出： 修订建议列表——"建议新增一条Skill处理时间格式转换"、"建议修改XX Prompt中的XX表述"。注意：**最终决定由人完成**——Dreaming只负责发现和建议，不直接修改。

为什么Dreaming和同步评测是互补而非替代?

- 同步评测：解决"单次任务做得好不好"——快速反馈，针对性强
- Dreaming：解决"跨任务的系统性模式"——全局视角，发现慢变量

类比： 同步评测像是"每次作业批改"——能抓住具体的错误。Dreaming像是"期中总结"——退一步看全貌，发现"这个学生数学都没问题，但语文理解能力系统性偏弱"。

实操建议：

- Dreaming频率：每天或每周运行一次（取决于会话量）
- 输入：最近N天的完整会话轨迹（包括成功和失败的）
- 输出：结构化的修订建议（带优先级和影响面评估）
- 处理：人工审阅后，高价值建议进入正式的进化流水线（走八环节）

3.5 落地避坑清单

高	评测缺失就上线	高	"自己试了两下觉得行"	改必测，测必过，门控全绿
高	Skill生成全自动	高	"AI生成应该没问题"	半自动：AI骨架+人审核
中	全文重写模式	中	LLM默认输出完整文件	Diff模式+框架patch合并
中	Skill无版本控制	中	"覆盖就好，旧的不要了"	Git化管理，每次有commit
中	无限循环	中	自改进进程没有终止条件	最大迭代次数+频率限制
!	配置层打转	低频致命	失败率不降一直改Prompt	3轮80%+不降→查工具层
!	方向池枯竭	低频	手写方向用完，进入平台期	Auto-Research引入外部知识

到这里，评测、记忆、落地三个环节都讲完了——Agent已经能自动发现问题、积累经验、产出修复方案并安全上线。但还有一个根本问题没有解决：**谁来确保这个自动运转的系统不会在错误方向上越跑越远？** 技术层面的门控能防住单次回归，但防不住缓慢的方向偏移。这就是最后一个齿轮——人机协作——要解决的事。

四、控制——人不是效率的瓶颈，是正确性的保障

> *飞轮的第四齿。完全自主的进化可能跑偏，完全人工驱动又太慢。关键是找到平衡点。*



4.1 我的核心论断：人的角色是教练和裁判，不是操作者

很多人把人机协作理解为"要么完全放手要么事事把控"——这是错误的二选一。

我的观点：**按场景分级自主，渐进放权，动态升降级。** Agent在稳定领域高度自主，在新领域和高风险操作必须有人。关键不是"人参不参与"，是"人在什么节点、以什么方式参与"。

为什么完全放手是危险的？ Anthropic《When AI builds itself》2026.6警告：AI在自主迭代修改自身配置时，可能导致行为发生**隐蔽且累积的偏移**——即使每次单独的改动看起来合理，累积起来可能已经偏离初始意图。在Harness层同样成立——而且由于飞轮在自动转，**一个错误被放大的速度和飞轮转动的速度成正比。**

4.2 分级自主模型



- 降级触发：出现P0回归→自动降Level 1直到修复
- 不可逆红线：安全边界/拒答逻辑/付费相关变更永远不能Level 3

4.3 五个必须有人的关键节点



无论Level多高，这五个节点**永远**不能交给Agent。不是"当前技术做不到"的权宜之计——而是**系统设计上的刚性约束。**

① 规则级记忆写入前。

规则级记忆（"遇到这类问题永远用方案A"）和普通记忆（"上次用了方案A效果不错"）有本质区别——规则级记忆会被无条件遵循，影响面是全局的。一条错误规则进入集群记忆后，所有后续任务都会遵循它——而且由于Agent"遵循规则"时不会再质疑规则本身，这条错误可能长期不被发现。**写入前必须人工确认——成本低（每次几秒钟看一条），但避免的损失巨大。**

② Prompt/Skill更新的最终确认。

特别是涉及安全边界、拒答逻辑、权限控制的改动。Agent不应该有能力修改自己的安全约束——**就像程序不应该有权限修改自己的权限配置文件。** 这是一个设计原则上的约束，不会因为技术进步而改变。即使Agent已经证明在其他方面完全可信——安全边界的修改权仍然只能在人手上。

③ 评测发现回归时的决策。

回归意味着"原本能做对的事做错了"——这需要人来判断：回滚到哪个版本？修复的优先级有多高？是阻断所有后续进化还是只冻结这一块？这些决策涉及业务影响面的判断（"这个回归影响了多少用户？严重度如何？"），是Agent没有足够上下文来做的。

④ 新领域冷启动时的初始经验种子。

没有历史数据的新场景，飞轮无法自动转起来——需要人提供第一批评测case（告诉系统"什么叫好"）、初始Skill（给出"大致怎么做"的方向）、安全边界（画清"什么绝对不能做"的线）。这是"手推飞轮第一圈"的动作——之后飞轮可以自己转，但第一圈必须人推。**试图跳过冷启动直接让Agent自己摸索，大概率的结果是在错误方向上快速迭代——越转越偏。**

⑤ 安全边界的设定与调整。

Agent能自主修改自己到什么程度？能改Prompt但不能改安全规则？能新增Skill但不能删除已有Skill？能修改记忆但不能修改评测集？**——这些边界线必须人画。** 如果让Agent自己决定自己的权限边界——就会出现Anthropic报告中警告的"隐蔽且累积的偏移"：Agent逐渐扩大自己的权限范围，每一步扩张看起来都合理，但累积起来可能已经失控。

4.4 审核疲劳：落地中最常见的失败模式



理想：关键节点有人审。现实：**什么都弹审核，人三天后不看直接通过**——比没有审核更危险，因为它给了"有人审了"的虚假安全感。

审核疲劳的本质原因：**到人手上的东西质量太差、数量太多。** 如果每天收到50个审核请求，其中48个是明显可以自动过滤的低质变体——人的注意力被垃圾请求耗尽，等真正需要人工判断的那2个到来时，已经没有精力认真看了。

解法三层，核心思路是**减少审核数量、提高审核质量、让审核本身也能"进化"**：

层级一：自动化前四层门控，把95%的低质变体在到人之前就过滤掉。

语法检查、回归测试、统计显著性、Playbook一致性——四层全自动。能到人手上的东西已经是"通过了所有自动化检查、大概率有效"的候选。人的注意力只用在"机器判断不了、需要人类语义理解"的最后一步。

层级二：批量异步审核，而非逐条实时弹窗。

不是每次改动都立刻请求审核（这会打断人的工作）——而是攒一批（比如一天或一个迭代周期的产出），一次性呈现给人。呈现时附上：

- 每个候选的评测数据（通过率、相比基线的提升幅度）
- 涉及哪些case、影响面多大
- 2-3个代表性的输入输出对比示例
- AI的推荐理由（为什么认为这个改动有效）

让人做**批量选择题**——"这5个候选里，你觉得哪些可以上？"——而非"这个改动你批不批？再来一个，你批不批？再来一个……"。批量处理效率高（一次看完5个比分5次看完高效得多）、上下文完整（能对比不同候选的优劣）、决策质量好（不会因为第48个请求而走神）。

层级三：渐进放权+自动降级——让审核制度本身也能进化。

不需要永远Level 1。一个场景如果连续稳定运行3个月（评测通过率>95%、无P0回归、人工抽检持续无问题）——可以申请升到Level 2（人只事后抽检，不再逐条审批）。升级后如果出现回归——自动降级回Level 1，不需要人手动操作。

本质上这是在让审核制度根据历史表现自适应——信任通过实践积累，但一旦违反就立刻收回。

4.5 更深层问题：进化方向如何与人类意图对齐？

前面讨论的都是"技术层面的控制"——门控、回滚、审核。但还有一个更深层的问题：**Agent的自进化方向如何确保和人的长期意图一致？**

这个问题为什么重要？举一个具体例子：

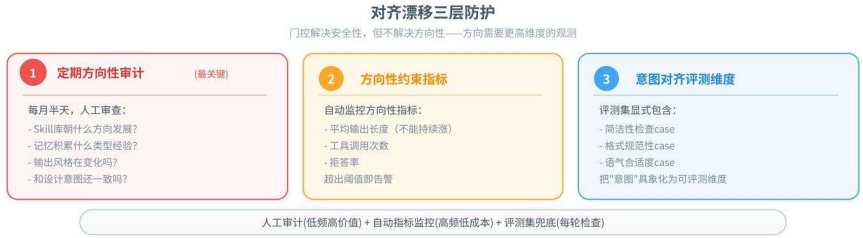
你希望Agent"回答准确且简洁"。但评测信号无意中奖励了"详细"——因为详细的答案更容易包含正确内容，导致评测分更高。多轮进化后，Agent变得越来越详细、越来越啰嗦——**每一步看起来都"在变好"（评测分确实在涨），但累积起来方向偏了（偏离了"简洁"这个意图）。**

版本？都不知道从哪个开始调的。

为什么技术门控抓不住对齐漂移？ 因为门控检查的是"这次改动有没有引入回归"（局部视角），不检查"这次改动是否让整体方向偏了一小步"（全局视角）。每一步都能通过门控——但步步偏移。

> **类比：** 想象你在迷雾中走路。每一步都检查了"这一步有没有掉进坑里"（门控通过了）。但没有人检查"这一百步加在一起，你还朝着目的地在走吗？"——可能你已经偏了90度，正在离目的地越来越远。**门控解决安全性，但不解决方向性。方向需要更高维度的观测。**

我的解法是三层防护：



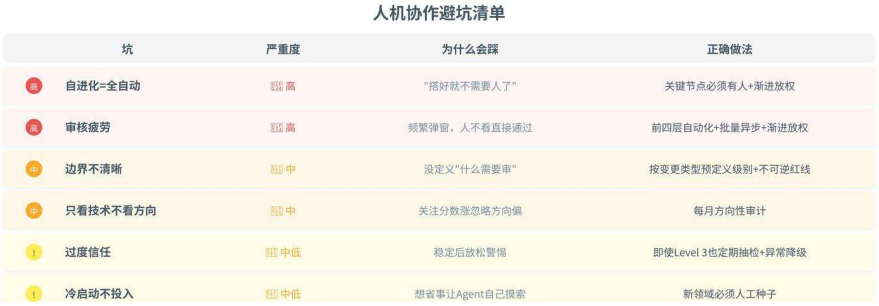
① 定期方向性审计（最关键）。 每月花半天时间，人工审查Agent最近一段时间的进化轨迹：

- Skill库在朝什么方向发展？新增了哪类Skill？哪类Skill被频繁修改？
- 记忆系统在积累什么类型的经验？有没有某类经验异常多？
- Agent的输出风格在变化吗？（更长了？更保守了？更激进了？）
- 这些变化和最初的设计意图还一致吗？

② 设置方向性约束指标。 除了"任务成功率"这种结果指标，还要监控一些**方向性指标**——比如平均输出长度（不能持续增长）、工具调用次数（不能越来越多）、拒答率（不能越来越高或越来越低）。这些指标的阈值由人设定，超出阈值就告警。

③ 评测集中显式包含"意图对齐"维度。 不只评"答案对不对"，还要评"答案风格对不对"——有几个评测case专门检查"简洁性"、"格式规范性"、"语气合适度"。把抽象的"意图"具象化为可评测的维度。

4.6 人机协作避坑清单



四个齿轮各自讲完了。但正如开篇所说——拆开来做每一块都能说"我做了"，关键是它们怎么咬合成一个持续转动的飞轮。最后一章，我们把四个齿轮装到一起。

五、飞轮——四个齿怎么咬合起来持续转动？

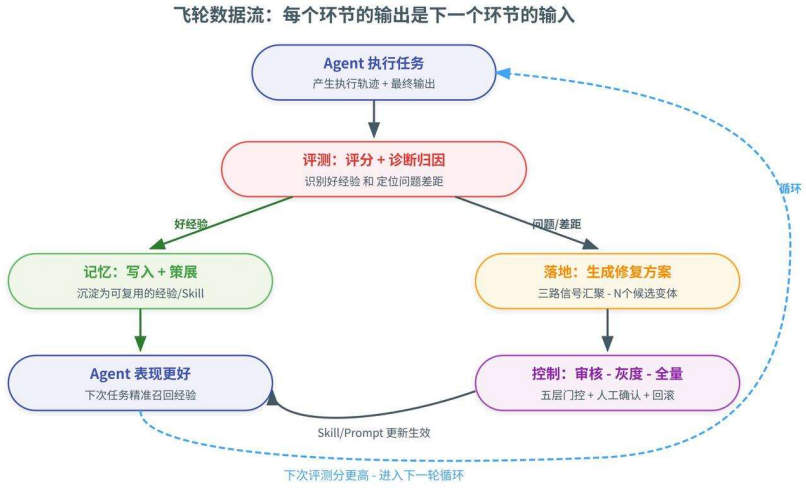


前四章分别讲了信号、积累、落地、控制。但**拆开来做每一块都能说"我做了"，合在一起却不产生复利**——这是开篇提出的核心问题。

现在回到这个问题：**它们怎么咬合成一个持续转动的飞轮？**

5.1 四个齿之间的数据流

飞轮能转起来的前提是：**每个环节的输出，恰好是下一个环节的输入。** 任何一条数据通路断了，飞轮就卡住。



具体来说，四条关键数据通路是：

通路一：评测 → 记忆。 评测产出"好经验"信号——哪些执行结果是好的、可复用的。这些正信号触发记忆系统的写入（策展式存储，不是全存）。如果这条通路断了——Agent做对了事情但不会记住，下次还是从零开始。

通路二：评测 → 落地。 评测产出"问题/差距"信号——哪些维度表现不好、失败模式是什么。这些诊断信号进入落地环节，驱动修复方案的生成。如果这条通路断了——Agent知道自己有问题但不会去改。

通路三：落地 → 控制 → 生效。 生成的修复方案经过五层门控和人工确认后灰度上线。如果这条通路断了——修复方案生成了但永远停在"候选"状态，不会真正影响Agent行为。

通路四：生效 → 下一轮评测（回流）。 更新生效后Agent的新表现会被下一轮评测捕获——成功了则正例加入金标集，新出现的失败则回流为下一轮种子。**这条通路是飞轮持续转动的关键**——没有它，进化是一次性的；有了它，每转一圈都比上一圈有更多信息。

大多数团队的问题恰恰在于：每个组件内部做得还行，但组件之间的"箭头"没有自动化。 评测跑完了但结果在报告里——没有流入Skill更新链路；记忆存了但没人回来验证存的对不对——没有闭环

5.2 飞轮的启动与持续

飞轮有一个冷酷的物理特性：**静止的飞轮需要很大的力才能推动第一圈，但一旦转起来，维持转动的力就小得多。** 自进化系统同理。

启动能量（人工推力——不能省、不能跳过）：

- **高质量的初始评测集。** 这是飞轮的"眼睛"——如果第一批评测集质量差（覆盖不全、标准不清、难度不合适），整个系统从第一天就会往错误方向跑。值得花1-2周精心设计。
- **人工注入的第一批 Skill/记忆。** Agent在没有任何经验积累时处于"白板"状态。人需要提供最初的几条核心Skill（"遇到这类问题大致这样处理"）和基础事实（"我们的系统架构是这样的"），给飞轮一个初始方向。
- **清晰的安全边界设定。** 在Agent开始自主进化之前，必须先画清"什么绝对不能做"的线。这条线越早画越好——因为一旦飞轮转起来，错误的扩散速度和飞轮转速成正比。

持续动力（自动回流——让飞轮越转越快）：

- **线上新失败样本自动回流为下一轮评测种子。** 不需要人手动收集——系统自动把灰度期间和线上监控中发现的新问题导入进化流水线。这是飞轮的"燃料供应"。
- **成功经验按晋升规则自动升级为 Skill。** 临时记录(.learnings/) → 经多次验证 → 升级为正式Skill。不需要人判断"什么时候该升级"——满足晋升条件自动触发。
- **异步 Dreaming 进程持续发现跨会话模式。** 单次评测看不到的全局性问题，由Dreaming定期巡检发现。它的产出进入下一轮进化种子池。
- **Auto-Research 引入外部知识防止方向枯竭。** 当内部经验用尽后，系统自动搜索论文/开源项目引入新方向——防止飞轮在"用完已知方向"后停转。

5.3 一个重要观察：什么样的场景最容易跑通？



观察已经成功跑通全闭环的案例——工单自动修复场景（解决率从不到30%提升至近90%）、内容推荐场景（真实CTR驱动进化）、长期记忆增强场景（LongMemEval基准显著提升）——它们共享两个特征：

特征一：评测信号来自真实业务指标，而非人工构造。 用真实CTR、拒审率、工单修复成功率等线上指标做反馈信号。真实指标天然不存在评测集漂移问题——业务在变，指标跟着变。**如果你的场景有天然的业务指标可以做反馈信号——优先用它，这是飞轮最自然的燃料。**

特征二：闭环链路短、反馈延迟低。 从"评测发现问题"到"修复方案生效"的链路越短，飞轮转得越快，团队越快看到收益，越有动力继续投入。实践经验表明5轮迭代内就能看到显著提升——**如果你设计的系统需要跑20轮才能看到效果，大概率团队早就失去耐心放弃了。**

5.4 从零到一的落地路线



Phase 1：最小闭环（1-2 周）。

心目的不是自动化，而是验证闭环本身是否成立——如果手动都跑不通，自动化只会更快地暴露问题。

Phase 2：半自动化（2-4 周）。 Skill自动生成管线 + CI评测门禁 + 版本管理 + 评测集三分法。目标："能自动改进且不退化"。人工占比 40%。

Phase 3：飞轮运转（1-2 月）。 灰度发布 + 回流机制 + 异步Dreaming + 渐进放权。目标："飞轮能自主转动"。人工占比 15%。

Phase 4：元进化（持续探索）。 Auto-Research + 进化策略自优化 + 多Agent协同进化。目标：飞轮越转越快。人工占比 5%。

核心建议：先跑通 Phase 1。 一个能手动转一圈的粗糙飞轮，好过一个精美但静止的蓝图。很多团队的错误是——Phase 1 还没验证就开始搭 Phase 3 的基础设施，最后发现闭环本身就没跑通。

收束



写到这里，把全文的核心信息浓缩为**三条原则和一个行动建议**：

原则一：评测的可信度 > 系统的复杂度。

在搭任何自动化之前，先确保你的评测信号是准的。评测不可信的系统，自动化程度越高，犯错速度越快。一个用手动评测+手动改Skill的简单闭环，远好于一个评测失真但全自动的复杂系统。**先把"眼睛"擦亮，再让"手脚"动起来。**

原则二：记忆系统是治理问题，不是存储问题。

向量库不是记忆系统。能存东西不代表能治理好。版本控制、主动遗忘、冲突解决、来源溯源、晋升门控——这些"不性感"的治理能力才是决定记忆系统到底帮忙还是添乱的关键。**存100条低质记忆不如治理好10条高质量记忆。**

原则三：闭环的价值在于环节之间的衔接，而非单个环节的先进性。

评测信号必须能自动流进记忆和Skill更新链路；Skill更新后必须能被评测自动验证；验证结果必须能自动反哺下一轮进化。**这些环节之间的"箭头"——而非环节本身——才是飞轮能不能转的决定因素。** 大多数团队不是某个环节做得不好，而是环节之间的数据通路是断的。

> *自进化不是一个功能，是一种系统设计范式。*
> *它的核心不是"让 Agent 自动变聪明"——而是搭建一套工程闭环，让每一次执行都不白费，每一个教训都不白吃，每一次改进都经过验证。*
> *这件事最难的部分，不是任何一个技术环节，而是让四个齿轮咬合在一起，持续地转。*
> *现在，你知道该怎么让它转起来了。*

发布于2026-08-27 18:26

多模态大模型论文：从CLIP、BLIP到BLIP2

Trancy Wang，机器学习、NLP、多模态、大模型，信息系统与信息管理博士

Trancy Wang： 前言： 自然语言处理在大模型的加持下发生了天翻地覆的变化，但技术是不断前进的，仅仅在文字模态上已经远远不够了，所以多模态势必是一个大的趋势，所以本文解析了多模态领域比较经典的3篇论文： CLIP、BLIP、BLIP2 这几篇论文均聚焦于视觉语言模型领域，旨在提升模型在视觉语言任务中的性能与泛化能力... 阅读全文

▲ 赞同 59 ▼ 1 条评论 收藏 喜欢 分享 ···